# Manual Técnico

Marketplace SENA — Centro Agroindustrial Hachón

## 1. ¿Qué es este proyecto?

Es un **marketplace web** para el SENA Centro Agroindustrial Hachón. El cliente explora un catálogo de productos agroindustriales (huevos, lácteos, frutas, etc.), **aparta** productos por **1 hora** y luego va al **punto de venta (POS)** a pagarlos y recogerlos.

La web **no cobra en línea**. El dinero se maneja en el POS de escritorio (Python/Flet, repositorio aparte). Ambos sistemas comparten la base de datos MySQL **sena_pdv**: la web usa la columna limite_venta (stock online) y el POS usa stock (stock físico).

## 2. Actores y roles

| Rol | Qué puede hacer |
|---|---|
| cliente | Explorar, apartar, cancelar sus apartados, ver su perfil y sus notificaciones. |
| tecnico | Gestionar productos, stock, ofertas, imágenes, carrusel y pedidos. No ve clientes ni analítica por defecto. |
| supervisor / promotor | Acceso administrativo, salvo la matriz de permisos y la gestión de cuentas Administrador. |
| admin | Acceso total. Es el único que puede editar la matriz de permisos. |

### Ciclo de vida de un apartado

Es el corazón del sistema. Un apartado pasa por estos estados:

- **pendiente:** el stock online ya se descontó. Dura 1 hora.
- **confirmado:** el admin validó el pedido y el cliente puede pasar al POS a pagar.
- **entregado:** se marcó al entregar en el POS. **Solo los entregados cuentan como venta** en la analítica.
- **cancelado:** puede ocurrir en cualquier etapa (por el admin, por el cliente o por vencimiento). **Siempre devuelve el stock.**

**Caso especial de los huevos:** se venden por cubeta, pero se almacenan en unidades. 1 cubeta = 30 unidades. Si el cliente aparta 2 cubetas, se descuentan 60 unidades de limite_venta.

## 3. Stack tecnológico

| Área | Tecnología | Uso en el proyecto |
|---|---|---|
| Backend | Node.js 18+, Express 5 | Servidor, rutas y middlewares. |
| Vistas | EJS | Plantillas HTML renderizadas en el servidor. |
| Base de datos | MySQL 8+ con mysql2 | Pool de 10 conexiones, consultas parametrizadas. |
| Sesión y seguridad | express-session, helmet, express-rate-limit, express-validator, bcryptjs | Sesiones de 1 hora, cabeceras, límites de peticiones, validación y hash de claves. |
| Reportes | pdfkit, CSV | PDF de analítica y exportación CSV con separador «;» y BOM para Excel. |
| Frontend | JavaScript y CSS vanilla, Chart.js 4.4.1 (CDN) | Sin framework, sin bundler y sin paso de build. Chart.js solo en analítica. |

| Área | Tecnología | Uso en el proyecto |
|---|---|---|
| Pruebas | node:test y node:assert | 35 pruebas unitarias con npm test. |

*Nota: qrcode está en package.json pero no se usa en ningún archivo.*

## 4. Instalación y arranque

**Requisitos:** Node.js 18 o superior, MySQL 8 o superior y Git (opcional).

1. Clonar el repositorio e instalar dependencias: npm install.
2. Crear el archivo .env en la raíz con las variables de la tabla siguiente.
3. Crear la base de datos con el script integrado: mysql -u root -p < database/schema.sql.
4. Arrancar: npm start (producción), npm run dev (desarrollo con recarga) o npm test (pruebas).

El servidor queda en http://localhost:3000. El script siembra el usuario inicial **admin / admin123**. **Cámbiese la clave antes de cualquier uso real.**

| Variable | Propósito |
|---|---|
| BD_HOST, BD_PORT, BD_USER, BD_PASSWORD, BD_DATABASE | Conexión a MySQL. También acepta los nombres DB_*. Por defecto: localhost, 3306, root, base sena_pdv. |
| PORT | Puerto HTTP del servidor (por defecto 3000). |
| SESSION_SECRET | Secreto de la cookie de sesión. Si falta, se usa un valor inseguro por defecto. |
| NODE_ENV | Con el valor production activa cookies seguras y confianza en el proxy. |
| RATE_LIMIT_MAX | Peticiones máximas por IP cada 15 minutos (300 por defecto). |

**Validar un cambio antes de darlo por bueno:** node --check sobre cada JavaScript modificado, node -e "require('./app.js')" para confirmar que la app carga y npm test si se tocan utilidades o CSRF.

## 5. Arquitectura

El proyecto sigue un patrón en capas, sin framework MVC: **rutas → middlewares → controllers → models → pool MySQL**. Los controllers concentran la lógica de negocio y los models solo ejecutan SQL. Cada respuesta puede ser una **vista** (HTML de EJS) o una **API** (JSON con la forma { ok, mensaje, ... }). Muchas vistas del panel son esqueletos que el JavaScript llena después con fetch.

Los controllers se agrupan por dominio en subcarpetas: admin/, cliente/, sitio/, carrusel/ e historicos/. Al arrancar, server.js dispara tres servicios: el job que expira apartados (cada 60 segundos), la carga de la matriz de permisos a memoria y las migraciones que agregan columnas y tablas faltantes.

**Orden de middlewares** para cada petición. El orden es crítico y está definido en app.js:

1. Cabeceras de seguridad (helmet) y compresión gzip.
2. Archivos estáticos de public/.
3. Helpers de rol puros disponibles en las vistas.
4. Parsers de cuerpo con límite de 4 MB (para imágenes en base64).
5. Límite general de peticiones por IP.
6. Sesión y permisos del usuario en res.locals.
7. Token CSRF y contador de visitas.

8. Verificación CSRF: cualquier POST, PUT, PATCH o DELETE sin token válido responde 403.
9. Rutas (públicas, luego cliente y luego admin), 404 y manejador de errores.

## 6. Estructura del repositorio

| Carpeta / archivo | Contenido |
|---|---|
| server.js, app.js | Arranque del proceso y construcción de Express. |
| config/ | Conexión MySQL, opciones de sesión y límites de peticiones. |
| database/ | Pool de conexiones y schema.sql (21 tablas, web y POS). |
| services/ | Job de expiración, caché de permisos, auditoría y migraciones al arrancar. |
| models/ | Consultas SQL (11 archivos). Ningún archivo contiene lógica de negocio. |
| controllers/ | Lógica de negocio, organizada por dominio. |
| routes/ | Definición de URLs: publico.js, cliente.js y admin.js. |
| middleware/ | Guardas: CSRF, sesión, roles, permisos, visitas, noCache y validación. |
| utils/ | Funciones puras: roles, permisos, cálculo de ofertas e imágenes. |
| test/ | Cuatro archivos de pruebas con 35 casos. |
| public/ | CSS, JavaScript del navegador, vistas EJS, imágenes y documentación. |

## 7. Base de datos

El script database/schema.sql crea la base **sena_pdv** con 21 tablas que son compartidas entre la web y el POS. Es idempotente en gran parte, pero no del todo (ver sección 12). Las tablas más importantes para la web son:

| Tabla | Función y notas |
|---|---|
| usuarios | Personal del panel y cajas del POS. La web agrega la columna activo. |
| clientes | Compradores. El documento es el usuario de inicio de sesión. La clave puede ser NULL. |
| producto | Catálogo compartido. stock es físico (POS) y limite_venta es online (web). |
| apartados | Reservas. El estado es un ENUM. precio_aplicado congela el precio con oferta al momento de apartar. |
| notificaciones, notificaciones_cliente | Campana del admin y notificaciones de cada cliente. |
| permisos_rol | Matriz de permisos por rol. |
| categorias, auditoria, page_views | Lista canónica de categorías, bitácora de acciones y visitas de los últimos 30 días. |
| contacto_recuperacion | Teléfono público para recuperar claves. No está en schema.sql: se crea al arrancar. |

**Trampa estructural:** apartados.nombre_cliente guarda el nombre como texto, no como clave foránea. La unión con clientes se hace por nombre. Por eso, al renombrar a un cliente hay que actualizar sus apartados, las notificaciones se buscan por nombre y dos clientes con el mismo nombre exacto pueden confundir datos. Las tablas contenido_sitio y banners son restos de un mini-CMS eliminado y no las usa la app.

## 8. Reglas de negocio

- **Apartar un producto:** se valida la sesión, que el producto esté activo y que haya stock. Los huevos se multiplican por 30. El precio con oferta se calcula y se congela. Luego se descuenta el stock con un UPDATE atómico (WHERE limite_venta >= ?). Si ese descuento falla, el apartado recién creado se cancela.
- **Apartar el carrito (lote):** máximo 50 ítems. Primero se valida todo sin escribir. Si algo falla durante la escritura, se revierte lo ya creado.
- **Agotado:** cuando limite_venta llega a 0, el producto pasa a inactivo y sale del catálogo.
- **Cancelar devuelve stock:** toda cancelación (admin, cliente o expiración) suma de nuevo el stock y reactiva el producto.
- **Vencimiento:** el job del servidor cancela cada 60 segundos los pendientes con más de 1 hora. El frontend muestra un contador y también cancela al llegar a cero, como segunda red de seguridad.
- **Ofertas:** el descuento va de 0 a 99,99. La fecha de inicio no puede ser posterior a la de fin. Las fechas vacías significan sin límite.
- **Analítica:** solo cuenta como venta el estado entregado. Las visitas excluyen el tráfico del panel y las rutas de API.
- **Protección de cuentas:** quien tiene el permiso de usuarios no puede gestionar cuentas Administrador. Nadie puede editarse a sí mismo y siempre debe quedar al menos un admin activo.
- **Acceso del cliente:** el login es de tres pasos (documento, clave y sesión). Si el cliente no tiene clave, debe pedirla al administrador. La clave temporal de recuperación se muestra una sola vez y solo se guarda su hash.

## 9. Seguridad

- **Cabeceras y límites:** helmet con política de contenido (CSP). Límite de 300 peticiones por 15 minutos por IP y de 5 intentos cada 15 segundos en login y registro.
- **Sesión:** cookie httpOnly y sameSite lax, de 1 hora de duración. Secure solo en producción.
- **CSRF propio:** token de 48 caracteres hexadecimales por sesión. El frontend lo envía en la cabecera x-csrf-token.
- **Claves y consultas:** bcrypt para todas las claves y SQL parametrizado en todas las consultas.
- **Roles en el servidor:** cada ruta tiene su propio middleware. Los controllers revalidan en varias pantallas.
- **Respaldo sin secretos:** la exportación usa una lista blanca de tablas y nunca incluye columnas de contraseña.
- **Ruta de archivos:** al eliminar imágenes del carrusel se rechaza cualquier nombre con rutas de directorio.
- **Auditoría:** las acciones sensibles del panel quedan registradas con usuario, rol, acción, detalle y fecha.

## 10. Endpoints principales

| Área | Rutas principales | Protección |
|---|---|---|
| Público | /, /catalogo, /producto/:id, /login, /registro, /api/productos | Sin sesión. Login y registro con límite de intentos. |
| Cliente | /perfil, /verApartados, /api/apartar-producto, /api/apartar-lote, /api/apartados/cancelar/:id, /api/mis-notificaciones | Sesión de cliente. |
| Pedidos | /admin/pedidos, /api/admin/apartados?estado=, confirmar, entregar y cancelar | Permiso pedidos. |

| Área | Rutas principales | Protección |
|---|---|---|
| Productos y ofertas | /admin/habilitar-producto, /admin/ofertas y sus APIs | Permisos productos y ofertas. |
| Clientes y carrusel | /admin/clientes, /admin/carrusel y sus APIs | Permisos clientes y carrusel. |
| Analítica | /admin/historicos, /api/admin/historicos/*, reportes CSV y exportar PDF | Permiso analitica. |
| Gestión interna | Usuarios, categorías, auditoría, respaldo y recuperación | Permiso del mismo nombre. |
| Permisos | /admin/permisos y /api/admin/permisos | Solo el rol admin. |

El orden de declaración importa en Express: las rutas con segmentos literales (como precio o activo) deben ir antes de las que usan el comodín :id.

## 11. Frontend

Los scripts son globales y se cargan con <script> en orden fijo: csrf.js, luego toast.js, luego login.js y finalmente el módulo de la página. El archivo csrf.js debe ir primero porque parchea fetch para añadir el token. El carrito público se guarda en localStorage. Los mensajes al usuario se muestran con toast() o con cajas #mensaje, según la vista. Los módulos del panel viven en public/js/admin/.

## 12. Pruebas e integración con el POS

npm test ejecuta 35 pruebas unitarias con el runner nativo de Node. Cubren roles, cálculo de ofertas, permisos y el token CSRF. No hay pruebas de integración con la base de datos ni de controllers.

Web y POS se coordinan así: el cliente aparta en la web, el admin confirma, el cliente va al POS, el POS registra la venta y la vincula al apartado mediante venta.id_apartado, y el apartado queda como entregado. **Antes de tocar schema.sql o las tablas compartidas, hay que considerar ambos sistemas.**

## 13. Cómo agregar funcionalidad nueva

1. Definir la regla de negocio y escribir el SQL en un model, con consultas parametrizadas y el mismo estilo (callbacks o async) del archivo.
2. Crear el controller en controllers/<dominio>/. Validar la entrada, responder con el código HTTP correcto y registrar las acciones sensibles con auditoriaService.registrar.
3. Agregar la ruta en admin.js, cliente.js o publico.js con su middleware. Si es un módulo nuevo, agregar su clave a utils/permisos.js y sus pruebas.
4. Crear la vista EJS en public/views/ con el meta CSRF y los scripts en el orden correcto.
5. Escribir el JavaScript del módulo con el patrón try/catch, respuesta.json() y toast(). Usar escaparHTML() cuando se inserte texto en innerHTML.
6. Verificar con node --check, node -e "require('./app.js')" y npm test.

**Convenciones del repositorio:** comentarios en español y en primera persona, con un bloque inicial que explique qué hace el archivo y por qué. Identificadores en español sin tildes (camelCase en JavaScript, snake_case en SQL). Validaciones con return temprano. No mezclar callbacks y promesas dentro del mismo archivo. No usar emojis en el código.

## 14. Deuda técnica y advertencias

Estos puntos están marcados en el código o se detectaron en la revisión. Conviene conocerlos antes de modificar el sistema:

1. **Condición de carrera al apartar:** el apartado se crea antes de descontar el stock. La solución correcta es una transacción con SELECT ... FOR UPDATE.
2. **Reversión de lote incompleta:** al revertir un carrito por falta de stock, no se devuelve el stock del último ítem.
3. **XSS:** verApartados.js y mainAdmin.js insertan notificaciones con innerHTML sin escapar.
4. **Más vendidos:** obtenerDestacados puede mostrar productos inactivos porque no filtra por estado.
5. **Asociación por nombre:** las imágenes de producto y la detección de huevos dependen del nombre. Si el nombre cambia, la imagen se pierde.
6. **Sin transacciones SQL:** la integridad depende de UPDATEs atómicos y compensaciones manuales.
7. **Secreto de sesión:** si falta SESSION_SECRET, se usa un valor inseguro por defecto.
8. **Limpieza de visitas:** contadorVisitas borra registros de más de 30 días en cada petición. Debería ser un cron diario.
9. **Esquema no totalmente idempotente:** algunos CREATE INDEX y ADD CONSTRAINT fallan si se ejecuta el script dos veces.
10. **Códigos HTTP inconsistentes:** varias validaciones responden 200 con {ok:false} en lugar de 400.
11. **Código duplicado:** la generación de CSV, varios helpers del frontend y la conversión de huevos aparecen en más de un lugar.

## 15. Documentación y código: diferencias

- El README no lista varios endpoints que sí existen, como POST /api/apartar-lote, las rutas de notificaciones del cliente y GET /api/admin/historicos/exportar.
- El rol aprendiz sobrevive solo como **tipo de cliente** en el registro. Ya no es un rol del panel.

## 16. Glosario

| Término | Significado en el proyecto |
|---|---|
| Apartado | Reserva de un producto por 1 hora. Es la unidad central del sistema. |
| Cubeta | Empaque de 30 unidades de huevo. La interfaz habla en cubetas y la base de datos en unidades. |
| limite_venta / stock | Stock online (web) y stock físico (POS), ambos columnas de producto. |
| precio_aplicado | Precio con oferta congelado al momento de apartar. |
| Permiso | Clave de módulo (productos, pedidos, etc.) que el admin delega por rol. |
| Canonizar rol | Normalizar variantes de la BD (por ejemplo, «Administrador») a su forma canónica (admin). |
| Esqueleto EJS | Vista que el servidor renderiza vacía y que el JavaScript llena con fetch. |